Ali User Manual

Ali User Guide

Current Bootstrap

Ali currently opens as a WPF desktop app with Chat as the home base.

You can:

- Type in the composer.
- Press Enter to send.
- Press Shift+Enter for a new line.
- Paste a copied screenshot or image with `Ctrl+V`.
- Use `Paste Image` when the clipboard contains an image.
- Use `Capture Screen` to attach a full-screen screenshot.
- Preview and remove image attachments before sending.
- Check `Retain` on an attachment only when you want Ali to keep it after the current send.
- Use `Push to Talk` to start local voice recording.
- Use `Stop Recording` to stop recording and ask Ali to transcribe locally.
- Review or edit the transcript before clicking `Send Transcript`.
- Use `Stop Speaking` to stop local spoken playback.
- Stop an active response.
- Flag an assistant answer as incorrect.

The first runtime is a safe local bootstrap stub. It exists to prove the app flow and correction queue before a real local model is activated.

Program Flow

This is the current V1 shape from a user's point of view: chat starts in the cockpit, runtime activation goes through Settings, local answers stream back into chat, and corrections/memory/reminders stay local. Voice is shown as local-only but not live-certified yet.

Local Runtime Check

Ali can now be pointed at a local OpenAI-compatible runtime, such as Ollama running on this PC.

Ali does not silently switch. Use the Runtime Settings panel:

1. Set the endpoint, usually `http://127.0.0.1:11434/v1/`.
2. Select the installed model/package ID exactly as the local runtime reports it.
3. Keep conservative development settings unless deliberately testing a larger profile.
4. Click `Check`.
5. Click `Activate` only after the check passes.

If the check fails, Ali keeps the safe bootstrap stub active and reports the failure.

Ali refuses public/cloud runtime endpoints in local-only mode.

Use `Revert to Stub` any time you want to return to the deterministic local test runtime.

The current certified local proof model is `qwen3-vl:8b` through Ollama's OpenAI-compatible endpoint. The development profile used for the latest certification was:

- Endpoint: `http://127.0.0.1:11434/v1/`
- Model: `qwen3-vl:8b`
- Quantization: Ollama package default / lowest Ali runtime settings
- Context: `2048`
- Output: `128`
- Temperature: `0`
- Top-p: `0.1`
- Streaming: enabled

`qwen3-vl:8b` can emit a separate Ollama reasoning stream before final answer content. Ali hides that reasoning in normal chat. If the model spends the whole low output budget on hidden reasoning, Ali reports that no visible assistant content arrived instead of exposing the reasoning text.

`qwen3:14b` was removed from this development machine to keep the system responsive.

HTML Helper

Ali also has a small web helper for basic ask/answer access.

Default local launch:

```
dotnet run --project .\src\Ali.App.WebHelper\Ali.App.WebHelper.csproj
```

Then open:

```
http://127.0.0.1:8765/
```

For remote or LAN testing, bind deliberately and use an access token:

```
$env:ALI_HELPER_URLS = "http://0.0.0.0:8765"
$env:ALI_HELPER_TOKEN = "choose-a-local-test-token"
dotnet run --project .\src\Ali.App.WebHelper\Ali.App.WebHelper.csproj
```

The helper serves one HTML page for basic chat, recent history, and runtime error reporting. It uses Ali's existing local runtime settings and tries to activate the configured local runtime on first ask. If the local runtime check fails, the helper returns the failure instead of silently using the stub as if it were the real model.

The helper lists the last 20 conversations from the current local Ali profile. This is not a personal-account or multi-user system; anyone with access to the same helper/profile can see the same helper history. If `ALI_HELPER_TOKEN` is configured, ask and history endpoints require that token.

The helper includes a local Programming Companion panel for deterministic Ali coding commands. The helper keeps recent chat history on the left, chat in the center, and the Programming Companion on the right. It starts with common entry points: What Can Ali Do, Plan A Build, Fix A Build, Install Check, VS Setup, and Windows Help. It then groups the builder flow as Goal, Options, Criteria, Tests, Roadmap, Next, Packet, and Validate. Deeper planning, execution, diagnostics, Git, and report commands remain below that flow. The bridge endpoints are loopback-only and still route through Ali's existing coding parser, workspace policy, confirmation gates, and receipts.

The helper does not add cloud fallback, voice, user-isolated accounts, or direct ungated file authority. Personal accounts and separated conversation history belong to a future hosted/multi-user product scope.

Audio Setup Sources

Ali includes a curated source catalog for the audio kit Chris plans to ship with the AI computer. The catalog points at official sources for the Focusrite Scarlett Solo/2i2, Audio-Technica AT2040, TritonAudio FetHead, and Shure Gator Low Profile Boom Arm SH-BROADCAST2.

This is source-backed reference material, not a special hardware setup feature. Ali may use it for ordinary questions about signal chain, drivers, phantom power, gain-staging procedure, mounting notes, and where to find official manuals. Ali should still ask for exact interface generation and avoid claiming one universal gain setting.

Coding Assistant

Ali can help with coding work inside the approved local programming workspace.

Current default workspace:

```
C:\Users\clsor\Documents\Programming Projects
```

Use Settings -> Permissions to review or change the approved workspace, file-open behavior, edit gates, build/test/run gates, and Git gates.

Useful commands:

- `inspect coding workspace`
- `show project map`
- `analyze solution architecture`
- `explore build idea SolidWorks BOM helper`
- `draft implementation roadmap Visual Studio tool window`
- `show pending roadmap`
- `approve last roadmap`
- `start approved roadmap`
- `show active roadmap step`
- `show next coding action`
- `show execution packet`
- `approve execution packet`
- `show packet commands`
- `run packet item 1`
- `confirm run packet item N`
- `show packet ledger`
- `show packet progress`
- `resume build plan`
- `mark roadmap step complete`
- `recover roadmap state`
- `show crash recovery status`
- `show visual studio integration`
- `generate visual studio integration plan`
- `interpret build goal screenshot bug helper`
- `show architecture options Visual Studio assistant`
- `write acceptance criteria package installer`

- suggest tests for VS companion
- detect codebase patterns
- plan feature files screenshot triage
- show refactor safety checklist command parser
- plan package lookup Visual Studio tool window
- plan dependency install packet QuestPDF
- preview project scaffold SolidWorks BOM helper
- plan scaffold apply SolidWorks BOM helper
- plan post edit validation
- show coding skill command index
- show coding session summary
- show computer assistant status
- show computer assistant commands
- what can you do
- can you tell me about your abilities
- what are your programming and data access limitations
- plan file organization "Downloads"
- plan disk cleanup
- plan app install troubleshooting Visual Studio installer crash
- plan peripheral setup Scarlett Solo gain
- show computer troubleshooting commands
- plan slow computer troubleshooting
- plan network troubleshooting
- plan wifi troubleshooting
- plan printer troubleshooting
- plan audio troubleshooting
- plan microphone troubleshooting
- plan camera troubleshooting
- plan bluetooth troubleshooting
- plan usb device troubleshooting
- plan display troubleshooting
- plan windows update troubleshooting
- plan app crash troubleshooting
- plan startup cleanup
- plan browser troubleshooting
- plan onedrive sync troubleshooting
- plan backup strategy
- plan driver troubleshooting
- plan suspicious activity check

- `plan remote support handoff`
- `show windows troubleshooting toolkit`
- `plan rogue process hunt port 8765`
- `collect process evidence dotnet`
- `diagnose port 8765`
- `diagnose build lock`
- `classify last build failure`
- `show roadmap step checklist`
- `show install doctor`
- `list packages`
- `confirm dotnet add package "CommunityToolkit.Mvvm" to "C:\path\to\project.csproj"`
- `search workspace for WidgetFactory`
- `read file "C:\path\to\file.cs" at line 42`
- `plan coding task fix the build`
- `show coding receipts`
- `suggest patch from last failure`
- `generate pdf "owner-demo.pdf" with text "Ali demo ready."`
- `show pdf commands`
- `inspect pdf "document.pdf"`
- `extract text from pdf "document.pdf"`
- `summarize pdf "document.pdf"`
- `convert markdown to pdf "notes.md" "notes.pdf"`
- `confirm combine pdfs "first.pdf" "second.pdf" "combined.pdf"`
- `generate install report pdf`
- `generate troubleshooting report pdf`
- `generate coding report`
- `generate morning report`

Guarded patch workflow:

1. Ask Ali to plan the task.
2. Ask Ali to preview the exact text change:

```
preview replace in file "C:\path\to\file.cs" "old text" with "new text"
```

3. Review the preview.
4. Use `show pending patch preview` if you need to see the pending patch again.
5. Use `discard pending patch preview` if the patch is not wanted.
6. Use `confirm apply last patch preview` to apply the last valid preview.
7. Use a confirmed build/test command after the edit.

Ali rechecks the pending patch before showing or applying it. If the file changed and the preview is stale, Ali discards the pending patch instead of applying an old change.

Multi-file patch bundles use the same preview and confirmation flow:

```
preview patch bundle
file "C:\path\to\first.cs" replace "old text" with "new text"
file "C:\path\to\second.cs" replace "old text" with "new text"
```

Patch bundles are limited to exact literal replacements inside the approved coding workspace. Ali allows up to eight edits in a bundle, including multiple sequential edits in the same file. Ali rechecks every edit before applying anything and writes only after the whole bundle validates. Use `confirm apply last patch preview` only after reviewing the preview.

Build and diagnostic workflow:

- Build/test/run/restore/package-install commands require confirmation.
- Use `confirm dotnet add package "Package.Id" to "C:\path\to\project.csproj"` to install a NuGet package into a project.
- Use `confirm dotnet build "C:\path\to\project-or-solution"` to run a guarded build.
- If a build or test fails, use `diagnose last build failure`.
- Use `open build error` to open the first diagnostic file at the reported line.
- Use `suggest patch from last failure` for narrow deterministic preview-only fixes. This currently supports simple `CS1002 ; expected` diagnostics by previewing a semicolon addition at the reported source line. It does not apply the change unless you review the preview and use `confirm apply last patch preview`.

Git workflow:

- Read-only commands such as `git status`, `git diff`, and `git log` are allowed when Git permissions are enabled.
- `git add`, `git commit`, and `git merge` require confirmation and follow the configured Git permission gates.
- Use `confirm git add all` and then `confirm git commit "message"` when you want Ali to write a commit.
- `git pull` and `git push` remain blocked unless network Git operations are deliberately enabled later.

Ali does not silently change files, run builds, restore packages, or write Git history. Tool results are recorded as coding receipts so Ali can report what actually happened.

Use `generate coding report` to export a local PDF summary of the current coding workspace, recent coding receipts, pending patch preview state, and last failed dotnet command if one is stored. The report is saved in Ali's configured PDF workspace.

PDF tools:

- `show pdf tool status` reports the configured PDF workspace, whether it exists, PDF permission gates, current capabilities, and current limits.
- `show pdf commands` lists the owner-facing PDF commands.
- `generate pdf "name.pdf" with text "..."` creates a polished local PDF with title styling, wrapped text, footer, timestamp, and page numbers.
- `inspect pdf "document.pdf"` reports file size, PDF version marker, page-count estimate, encryption/form/image markers, and text availability.
- `extract text from pdf "document.pdf"` writes extracted text to a `.txt` file in the PDF workspace when simple text is available.
- `summarize pdf "document.pdf"` produces a deterministic extractive summary from readable PDF text.
- `convert markdown to pdf "notes.md" "notes.pdf"` creates a polished PDF from a `.md`, `.markdown`, or `.txt` file.
- `confirm combine pdfs "first.pdf" "second.pdf" "combined.pdf"` creates a derived combined PDF from extractable text. It preserves originals and requires confirmation.
- `confirm split pdf "source.pdf" "split-output.pdf"` creates a derived split/extract PDF from extractable text. It preserves originals and requires confirmation.
- `generate install report pdf` and `generate troubleshooting report pdf` create focused owner-facing report PDFs.

PDF workspace and permission controls live under Settings -> Coding / Permissions. The PDF workspace has its own textbox and Choose Folder button. PDF inspect/extract, create/export, and combine/split/modify each have their own permission row. Ali is not a full Acrobat replacement yet; scanned/image-only PDFs, OCR, redaction, form editing, annotations, and layout-preserving arbitrary PDF editing are future work.

Computer assistant:

- `show computer assistant status` reports Ali's visible local assistant lanes, including coding/Visual Studio, PDF workspace, Windows troubleshooting, general computer planning, source-backed answers, and audio setup sources.
- `show computer assistant commands` lists everyday computer-help commands and routes plain questions such as `what can you do` and `can you tell me about your abilities` to the real ability index instead of letting the model guess.
- `plan file organization "Downloads"` creates a read-only organization plan and, when possible, a top-level folder snapshot. It does not move, rename, copy, or delete files.
- `plan disk cleanup` creates a read-only cleanup path and drive-space snapshot. It does not delete files or change Windows settings.
- `plan app install troubleshooting <app-or-error>` creates an installer troubleshooting plan without running installers, uninstallers, registry edits, driver installs, service changes, PATH changes, signing changes, or trust-store changes.
- `plan peripheral setup <device-or-symptom>` creates a setup plan for devices such as audio interfaces, microphones, and boom arms without changing drivers, firmware, default devices, exclusive-mode settings, services, or registry entries.
- `show computer troubleshooting commands` lists 20 read-only troubleshooting entry points for slow PCs, network/Wi-Fi, printers, audio/microphones/cameras, Bluetooth/USB/display, Windows Update, app crashes, startup cleanup, browser issues, OneDrive sync, backup, drivers, suspicious activity, and remote-support handoffs.
- `plan <scenario> troubleshooting` creates a scenario-specific evidence checklist and stop rules. These planners do not change Windows settings, install drivers, clear browser data, unlink sync providers, stop processes, or run repair tools.

Ali's source access is curated and approval-oriented. She can use approved source catalog entries when the app performs a source lookup, but this is not unrestricted browsing or autonomous web control. Spoken responses are cleaned so source appendices remain visible in text but are not read aloud by Piper.

Ability-index maintenance rule: whenever a feature is added, update `show coding skill command index`, `show computer assistant commands`, relevant Web Helper/VS Companion buttons, this guide, and the engineering notes so Ali can accurately describe her own current abilities.

`analyze solution architecture` is read-only. It reports solutions, projects, target frameworks, project roles, project references, package references, source-file counts, a project dependency graph, and an estimated build order.

`explore build idea ...` is read-only. It helps compare implementation paths, library/software areas to research, and approval checkpoints for a proposed build. Library names are exploration candidates only; Ali needs an approved internet/package lookup before treating versions or ecosystem state as current.

`draft implementation roadmap ...` is read-only. It expands a goal into architecture fit, phases, likely impact surface, test strategy, risks, definition of done, approval checkpoints, and safe next commands.

Roadmaps can be kept as pending planning state. Use `show pending roadmap`, `approve last roadmap`, `start approved roadmap`, or `discard pending roadmap`. Starting an approved roadmap begins a guided phase loop and proposes the next safe action. It does not silently edit files, install packages, run builds/tests, or write Git history.

Roadmap execution state is saved under Ali's local coding data. Use `show active roadmap step`, `mark roadmap step complete`, `pause roadmap`, `resume roadmap`, `finish roadmap`, or `recover roadmap state` to continue after interruption or crash. Recovery restores the roadmap goal, status, current step, and last receipt snapshot; it does not replay commands.

Use `show crash recovery status` after a crash or interrupted build. Ali reloads the roadmap state, checks recent coding receipts, runs a read-only Git status check, compares the active roadmap step against receipts, and suggests safe continue/fix/rollback paths. If the evidence supports a concrete fix, Ali can suggest a guarded patch preview for approval; if the evidence is unclear, she should pause and show options before editing.

Approved packet workflow:

1. Use `show execution packet` to review the next roadmap step.
2. Use `approve execution packet` to store that step as local planning state.
3. Use `show packet commands` to see numbered prep, execute, validate, and closeout commands.
4. Use `run packet item 1` for read-only items.
5. Use `confirm run packet item N` only when you deliberately want Ali to run a gated build, package, edit, run, or Git command from the packet.
6. Use `show packet ledger` and `show packet progress` to compare receipts against the approved packet.

Builder, package, and scaffold planning:

- `interpret build goal <goal>` classifies the requested build, identifies a likely first milestone, shows architecture option cards, and lists approval checkpoints.
- `show architecture options <goal>` compares several architecture paths before a build starts.
- `write acceptance criteria <goal>` drafts a "done means..." checklist for the feature.
- `suggest tests for <goal>` recommends focused parser, service, policy, UI, package, or screenshot/vision tests based on the goal.
- `detect codebase patterns` reports local project roles, package patterns, and implementation conventions so suggestions fit the repo.
- `plan feature files <goal>` lists likely files/classes to inspect or modify.
- `show refactor safety checklist <goal>` highlights contract, persistence, permission, UI, migration, and validation risks before edits.
- `plan package lookup <goal>` lists package/library exploration lanes, dependency risk cards, and the approval path for restore/outdated/package-install commands. It does not run internet or package-registry lookups by itself.
- `plan dependency install packet <goal>` turns dependency work into prep, approval commands, validation commands, and rollback notes. It does not install anything.
- `preview project scaffold <goal>` drafts a folder/file/test-project shape for a new feature. It does not create files or add solution entries by itself.
- `plan scaffold apply <goal>` shows how to move from scaffold preview to confirmed file creation and validation. It does not create solution entries by itself.
- `plan post edit validation` shows the next safe build/test/Git validation loop after approved edits.
- `show coding skill command index` lists Ali's programming powers, limits, and safest next commands.
- `show coding session summary` summarizes recent receipts, active roadmap/packet state, and useful next actions.
- `resume build plan` combines roadmap state, approved packet state, recent receipts, last dotnet failure, and Git status into the safest next step after a crash or interruption.
- `generate morning report` exports a PDF summary of packet commands, ledger state, resume guidance, dependency planning, scaffold planning, and install readiness.

Windows troubleshooting:

- `show windows troubleshooting toolkit` shows read-only PowerShell and CMD recipes for process, memory, port, service, startup, event-log, disk, network, and build-lock investigation.
- `plan rogue process hunt <target>` gives a focused plan for finding a suspicious process, locked file, or port owner.
- `collect process evidence <name-or-pid>` lists matching local process PID/name/path/start/memory evidence without stopping anything.
- `diagnose port <port>` runs a read-only `netstat -ano` check, maps the port to a PID when possible, and shows matching process evidence.

- `diagnose build lock` checks common build-lock suspects such as `dotnet`, `MSBuild`, `VBCSCompiler`, `Ali.App.WebHelper`, and `devenv`.
- `inspect services and startup` and `triage event logs` provide read-only PowerShell/CMD recipes for deeper Windows diagnosis.
- `plan stop process <pid>` stages an approval checklist. `confirm stop process <pid>` requests a normal `taskkill /PID <pid>` without a force flag.
- These commands are guidance only. They do not stop processes, disable services, edit startup entries, delete files, change registry/firewall/PATH/trust settings, or repair Windows.
- Process stopping and system repair actions require explicit owner approval with a named PID/process/service and a rollback note when applicable.

Build/install intelligence:

- `classify last build failure` labels the last stored dotnet failure as locked file, restore/package, compiler, test, missing SDK/tool, or unknown, then suggests safe next commands.
- `show roadmap step checklist` compares roadmap state, recent receipts, validation, and Git status before a roadmap step is marked complete.
- `show install doctor` reports install readiness for DevRun, Visual Studio discovery, VSIX artifact, current .NET runtime, workspace state, and related manual dependency checks. It does not run installers or repair anything by itself.

Visual Studio integration in this build now includes a developer VSIX named `Ali Companion`. The extension has Project Ali branding metadata, a packaged icon, and a native `Ali Companion` tool window in Visual Studio with command groups for Start Here, Awareness, Guided Flow, Plan Tools, Execute, Diagnostics, Reports, and PDF Tools. The Start Here buttons are What Can Ali Do, Plan A Build, Fix A Build, Install Check, VS Setup, Windows Help, and PDF Tools. The Guided Flow buttons are Goal, Options, Criteria, Tests, Roadmap, Next, Packet, and Validate. The PDF Tools group surfaces PDF status, command index, create, inspect, extract, summarize, Markdown conversion, combine, install report, and troubleshooting report commands. The tool window also includes command history, a command box, selected-code package preview, progress/state cues, run timing summaries, pending-approval summary, output tabs, and a Visual Studio context strip. Output is separated into Response, Diagnostics, Receipts, and Pending Patch tabs. It can read the active solution path, active document path, current line, and selected text, then fill deterministic Ali commands such as read active file, build active solution, search selection, plan from selection, and preview replace selection. The same staged commands are available from `Tools -> Ali Companion`, and the code editor context menu includes Ali entries for reading the current file, building the active solution, searching the selection, planning from the selection, and previewing a replacement for the selection. Solution Explorer context menus can stage Ali commands from selected solution, project, or file nodes: read selected file, build selected project/solution, or plan from the selected node. Visual Studio commands enable only when the needed active document, selected text, solution path, or Solution Explorer node is available. Selection-based commands show the exact packaged text, target, original length, packaged length, and trim status before the command is run. It calls Ali's loopback coding bridge at `http://127.0.0.1:8765/api/coding/*`; the helper still owns the coding command parser, workspace policy, confirmation gates, and receipts. Configure helper URL, history length, and selected-text behavior under `Tools -> Options -> Ali -> Companion`.

Use `generate visual studio integration plan` to produce a deterministic handoff for deeper phases. The handoff reports the current workspace, launcher discovery, architecture snapshot, and the minimum guarded contract the VSIX must keep following.

The HTML helper's Programming Companion panel remains available in the browser. The VSIX uses native Visual Studio controls instead of embedding that page, which avoids legacy embedded-browser rendering and script issues. Both surfaces submit deterministic coding commands to Ali on loopback, and all writes/builds/tests/Git actions still follow the same confirmation gates. Context buttons, editor right-click commands, and Solution Explorer node commands fill the command box; they do not silently edit files or run builds without Ali's normal command handling.

The VSIX build output is:

```
src\Ali.App.VisualStudioExtension\bin\Debug\net472\Ali.App.VisualStudioExtension.vsix
```

Install it into Visual Studio Community with Visual Studio's VSIX Installer, then open `View -> Ali Companion` or `Tools -> Ali Companion`. Start the WebHelper before using the window. The tool window can also open the browser helper

separately with `Open Helper`:

```
dotnet run --project .\src\Ali.App.WebHelper\Ali.App.WebHelper.csproj --no-build
```

Install and update notes for the native VSIX live at:

```
tools\visualstudio\ALI_COMPANION_VSIX_INSTALL_UPDATE_NOTES.md
```

Visual Studio can also call the current bridge through `Ali.App.VisualStudioBridge.exe`, which is designed for Visual Studio External Tools. Build the solution, start the WebHelper on loopback, then add an External Tool that points at:

```
src\Ali.App.VisualStudioBridge\bin\Debug\net10.0\Ali.App.VisualStudioBridge.exe
```

Example External Tools arguments:

```
--status
--handoff
--list-external-tools
--preset recovery
--preset build --solution "$(SolutionPath)"
--command "analyze solution architecture"
--read-current-file --file "$(ItemPath)" --line "$(CurLine)"
--command "read file \"{file}\" at line {line}" --file "$(ItemPath)" --line "$(CurLine)"
```

The External Tools bridge remains useful for one-click commands, even with the VSIX installed.

The full External Tools setup guide is at:

```
tools\visualstudio\ALI_VISUAL_STUDIO_EXTERNAL_TOOLS.md
```

The Visual Studio plugin polish backlog is tracked at:

```
docs\ALI_COMPANION_PROFESSIONAL_PLUGIN_BACKLOG.md
```

The bridge tries to start Ali's local WebHelper automatically when it is offline. Use `--no-start-helper` if you want Visual Studio to fail fast instead.

The bridge is not tied to Insiders. If you switch to regular Visual Studio Community later, add the same External Tools entries in that Community instance and point them at the same bridge executable.

Voice

Phase 1C voice is local-only.

Ali records a temporary WAV file from the selected local microphone. The raw audio is deleted after transcription unless a retention setting or validation flag explicitly keeps it.

The current local voice resources live under Ali's own `lib\voice` folder:

- `lib\voice\python-venv`
- `lib\voice\whisper`
- `lib\voice\piper`

For this developer build, configure the local speech environment from:

```
.\tools\voice\ALI_LOCAL_VOICE_ENV.example.ps1
```

Ali uses a local Faster-Whisper wrapper for STT and a local Piper wrapper for TTS. The STT wrapper writes confidence metadata and rejects suspicious no-speech/low-confidence segments instead of turning noise into a command.

Installed local proof resources: 26 en-US Piper voices and Faster-Whisper caches for `tiny.en`, `base.en`, `small.en`, `medium.en`, and `large-v3`.

If local STT or TTS is not configured, Ali says so in the voice status area. She does not use cloud speech and does not pretend a transcript or spoken response succeeded.

The voice settings popup includes:

- Microphone picker
- Channel picker

- Gain control
- Live input meter
- Capture diagnostics
- Piper voice picker and sample playback

The main chat surface is chat-first: conversation list on the left, conversation content in the center, and one bottom composer for typed text, image attachment, mic dictation, hands-free voice mode, stop response, and send.

The composer mic records local speech and places the accepted transcript into the chat bar. It does not send until Enter or Send is pressed. The voice mode button toggles hands-free behavior, where accepted transcripts are sent automatically. Risky command transcripts still require visible confirmation and are blocked in this phase.

Meter states:

- `No speech signal detected`: selected mic is silent, muted, or not receiving signal.
- `Input is too quiet`: Ali hears something, but STT may not be reliable.
- `Input level looks usable`: good candidate for live certification.
- `Input is clipping`: lower gain or choose a calmer preset.

Input presets:

- `Raw`: minimal processing.
- `Quiet Room`: light cleanup and moderate gain.
- `Noisy Room`: stronger gate/noise suppression.
- `Broadcast Mic / Close Mic`: close microphone shaping.
- `Headset Mic`: practical boosted default for headset-style mics.

Voice settings persist in `%LOCALAPPDATA%\Ali\BootstrapData\voice-settings.json`. If a saved mic disappears, Ali shows a warning and falls back visibly instead of silently pretending the same mic is still active.

Current voice certification status: live voice, microphone, Piper playback, and Stop Speaking are not certified in the current V1 runtime pass. Chris must be present for that hardware/audio certification. Logic-level safety tests exist, but that is not the same as live voice certification.

Voice can ask ordinary chat questions. Voice cannot yet run commands, change models, edit calendars, install things, delete memories, or do destructive actions. Those spoken requests are blocked in this phase and require visible typed confirmation later.

Important Truth Rule

Ali must not claim a model, command, build, test, reminder, calendar event, or file change succeeded unless there is evidence.

If Ali does not know, she should say so.

Correction Queue

Use `Flag as incorrect` when an answer is wrong or unsupported.

Ali preserves:

- The exact question
- The exact answer
- Model profile metadata
- Evidence status

- The correction category
- Voice transcript and local STT/TTS metadata when the answer came from voice

The original answer is not rewritten.

If the flagged answer used a screenshot or image attachment, Ali routes it as a screenshot/image misread correction.

Raw voice audio is not stored in the correction queue.

Coming Next

- Installer-managed signing/repair so Smart App Control accepts refreshed Ali binaries without manual certificate work
- Owner visual review
- Live voice/mic/Piper certification with Chris present
- Real local Whisper/Piper install picker
- Source/search controls
- Memory controls
- Backup and restore
- Simple installer with repair mode